Universidad San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ingeniería en Ciencias y Sistemas.

Segundo Semestre 2026




# ResiPark: Control de Accesos y Estacionamiento

## Manual Tecnico



| Nombre | Registro Académico |
|---|---|
| Victoria Yannarett García Gómez | 202502718 |



Guatemala 23 de Octubre, 2026

Manual Tecnico

1.Descripción

Este manual explica cómo está diseñado ResiPark con una estructura en Java y cómo gestiona los  residentes, vehículos, espacios de estacionamiento y eventos, sin el uso de librerías externas más que todo coordinadas por medio de hilos para la entrada y salida de los vehículos.

2. Objetivos

ResiPark hace una simulación de garita o acceso a un residencial con el registro de residentes y sus vehículos, recibir solicitudes de ingreso, asignar espacios disponibles, registrar eventos y permitir la salida de los vehículos. La interfaz está hecha en Java Swing y la información se maneja mediante archivos de texto, no mediante una base de datos.

3. Componentes del sistema

| Área | Utilidad |
|---|---|
| Modelo | Son los usuarios, vehiculo y EspacioParqueo. |
| Estructuras | Implementar nodos y estructuras enlazadas propias: listas, cola, pila y parqueo circular |
| Hilos | Es la atención de las garitas, cuando corresponde a la versión final. |
| Vista | La ventana y los paneles Swing que permiten operar y observar el sistema. |
| Persistencia | Leer y escribir residentes y vehículos en archivos de texto, si esta capa está separada en la versión final. |

4. Entidades principales

- Usuario / residente =Es la persona residente e incluye, ID, nombre, casa o lote, condición de socio y una lista propia de vehículos.
- Vehiculo =  Es un automóvil, motocicleta o pickup. Debe mantener una placa única, datos descriptivos y un estado de circulación por el sistema.
- EspacioParqueo = Representa un espacio físico del estacionamiento.
- Evento = Conceptualmente registra fecha y hora, tipo, descripción y garita.

5. Estructuras de datos

5.1 Lista doblemente enlazada de residentes

Uso de nodos con referencias, inserta al final y permite buscar por ID recorriendo desde la cabeza, el tamaño y la cabeza, y muestra el recorrido hacia delante. Deben estar implementadas y conectadas las operaciones de eliminación y recorrido hacia atrás. Permite avanzar en ambos sentidos sin reconstruir el recorrido. La búsqueda sigue siendo secuencial.

5.2 Lista simplemente enlazada de vehículos

La clase Lista Simple enlazada nodos mediante una referencia siguiente. El TXT muestra el recorrido y acceso a la cabeza. Cada residente debe tener su propia lista de vehículos. El límite de negocio es de cero a tres vehículos por residente; la validación debe ejecutarse antes de insertar, no solamente en la interfaz.

5.3 Lista circular del parqueo

Los nodos se enlazan de forma circular: el último vuelve a apuntar a la cabeza. Para buscar un espacio, el algoritmo comienza después del último asignado y recorre la lista hasta hallar uno libre o completar una vuelta.

6. Flujo de entrada y salida

Entrada: el usuario selecciona un vehículo que está FUERA o registra un visitante válido; el sistema cambia su estado, lo agrega a la cola y registra el evento. Una de las dos garitas toma el primer vehículo disponible, simula su atención y determina si puede asignarle un espacio. Si no hay espacio permitido, el vehículo debe volver a FUERA y quedar registrado como rechazado.

Salida: únicamente un vehículo puede solicitar la salida. El estado cambia, pero el vehículo conserva su espacio mientras espera. La garita de salida consume la cola, libera el espacio y devuelve el vehículo a FUERA. En GaritaEntrada disponible el vehículo se envía directamente a parqueo General.

7. Hilos y sincronización

El diseño requiere tres hilos con trabajo real: dos garitas de entrada que comparten una única cola y una garita de salida que atiende la cola de salida. La cola vacía debe realizarse dentro de un while y con wait(); al agregar un elemento se debe llamar notifyAll() o notify() desde el mismo monitor. Las operaciones que modifican el parqueo y el historial deben protegerse para evitar asignaciones dobles y pérdida de eventos. El tiempo de atención se implementa con Thread.sleep entre uno y tres segundos. Como Swing no es seguro para modificaciones desde hilos secundarios, cualquier actualización de componentes visuales debe enviarse mediante SwingUtilities.invokeLater.

8. Validaciones que debe aplicar el sistema

• ID de residente obligatorio, único y no modificable después del registro.
• Campos de texto sin el carácter vertical |, porque separa columnas en los archivos.
• Placa única en todo el sistema: vehículos registrados, visitantes, vehículos en cola y vehículos estacionados.
• Máximo de tres vehículos por residente.
• Tipo de vehículo restringido a automóvil, motocicleta o pickup.
• Un vehículo FUERA puede solicitar ingreso; un vehículo ESTACIONADO puede solicitar salida.
• Un residente no puede eliminarse si alguno de sus vehículos no está FUERA.
• No cambiar la condición de socio si alguno de sus vehículos no está FUERA.
• Visitante vinculado a un ID de residente existente; visitante solo temporal y no persistido.
• No permitir que residentes no socios ni visitantes ocupen espacios exclusivos de socios.

9. Cálculo de ruta y asignación de espacios

Un recorrido circular de búsqueda de espacio: se empieza en el nodo siguiente al último asignado y se avanza nodo por nodo hasta encontrar un espacio libre o regresar al punto de inicio.

10. Persistencia

El formato de residentes.txt con ID|NOMBRE|CASA|SOCIO y vehiculos.txt con PLACA|MARCA|MODELO|COLOR|ID_RESIDENTE. Al iniciar, todos los vehículos deben reconstruirse en estado FUERA y el parqueo debe iniciar vacío. Los visitantes y la pila de eventos no se guardan. Al cargar datos deben omitirse líneas inválidas, residentes inexistentes o registros que excedan el máximo de tres vehículos, dejando un evento explicativo.

11. Interfaz y actualización visual

En la interfaz son visibles el estado de los 150 espacios, distinguir libres y ocupados, mostrar la placa de los vehículos estacionados y reflejar las colas y la garita que atiende cada vehículo. La versión final debe contener los paneles de parqueo, residentes y vehículos, entrada, salida, visitantes e historial de eventos según la especificación completa del curso.

12. Decisiones de Diseño e Implementación

Se optó por no utilizar librerías de colecciones estándar como java.util.ArrayList o LinkedList para cumplir con los requerimientos de estructuras de datos fundamentales. El renderizado en Swing se realiza actualizando paneles dinámicos mediante re-pintado en el Event Dispatch Thread.